Ders 10 Genişletilmiş Notu: Armstrong Sayıları, Yakıt Tüketimi, Sabitler, Değişkenler ve Diziler

Ders 10 Genişletilmiş Notu: Armstrong Sayıları, Yakıt Tüketimi, Sabitler, Değişkenler ve Diziler

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Algoritma 6: Üç Basamaklı Armstrong Sayılarını Bulmak ve Saymak
- 2. Algoritma 7: Arabanın Yakıt Tüketimi
- 3. Sabitler, Değişkenler ve Diziler
- 4. Dizilerin Bellekte Saklanması
- 5. Algoritma 8: Başarılı Öğrencilerin Sırasını Bulmak
- 6. Alıştırma: 365 Günün En Soğuk ve En Sıcak Günü

1. Algoritma 6: Üç Basamaklı Armstrong Sayılarını Bulmak ve Saymak

Hoca derse, döngülerle örnek çözmeye devam edeceğimizi ve ardından dizilere giriş yapacağımızı söyleyerek başladı. İlk problem şuydu: Üç basamaklı Armstrong sayılarını bulan ve kaç tane olduklarını da söyleyen algoritmanın akış diyagramını çizin.

1.1 Önce Problemi Doğru Anlamak

Bir öğrenci bu soruyu daha önce yaptığımızı düşündü. Hoca bunun üzerinden en temel kuralı hatırlattı: Birinci işimiz problemi doğru anlamaktır. Daha önce çözülen problem, bize **verilen bir sayının** Armstrong sayısı olup olmadığını söylüyordu. Şimdi ise iki şey isteniyor: Üç basamaklı Armstrong sayıları hangileridir ve kaç tanedir?

> **Hocanın vurgusu:** İsteneni anlamak birinci işimizdir. Benzer görünen iki soru, farklı iki problem olabilir.

Tasarım kararları sınıfla soru-cevap yapılarak verildi:

- **Döngü tipi:** Bir öğrenci while önerdi, hoca bunu kabul etmedi. Adım sayısı belli olduğu için for kullanılır. Adım sayısının belli olmasının sebebi, üç basamaklı sayılara bakıyor olmamızdır.
- **Dışarıdan girdi:** Alınmaz, çünkü kontrol edilecek sayıları biz biliyoruz.
- **Döngünün sınırları:** Başlangıç 100, bitiş 999.

Hoca, bir değer gerektiğinde genelde onu kullanıcının verdiğini, ama burada olduğu gibi girdi gerektirmeyen özel durumların da oluşabileceğini söyledi.

1.2 Akış Diyagramı

Bir öğrenci basamakların nasıl ayrılacağını ve koşulların nasıl yazılacağını anlamadığını söyleyince çözüm birlikte kuruldu. Basamaklar, daha önce verilen sayı için yaptığımız DIV ve MOD işlemleriyle ayrılır. Tek fark, ayrılan sayının artık döngü indisi `I` olmasıdır. Her turda yeni sayımız `I` ile karşımıza gelir. Hoca, başka bir çözümdeki değişken adlarını (`A` gibi) tahtadaki çözüme uyarlamak gerektiğini de özellikle söyledi.

1. START.
2. `count = 0`. Sayaca ilk değer döngüden önce verilir.
3. For döngüsü: `I ← 100`, adım `1`, son değer `999`.
4. Proses kutucuğu:
   - `yuzler = I DIV 100`
   - `onlar = (I DIV 10) MOD 10`
   - `birler = I MOD 10`
   - `kup = yuzler*yuzler*yuzler + onlar*onlar*onlar + birler*birler*birler`
5. If `I = kup`:
   - True kolu: `I` yazdırılır, sonra `count = count + 1` yapılır.
   - False kolu: Hiçbir şey yapılmaz, kollar birleşir.
6. Döngü başına dönülür.
7. Döngüden çıkınca `count` yazdırılır.
8. STOP.

Çözüm sırasında true kolunda yalnızca sayının yazdırıldığı söylendi. Hoca başka bir öğrenciye "başka bir şey yapmayacak mıyız?" diye sordu ve eksik parça bulundu: Kaç tane sayı olduğunu istediğimiz için sayacı da artırmamız gerekir.

> **Ders sonrası not:** Altyazıda sayıların kendisi söylenmiyor. Hoca yalnızca "dört tane var" diyor. Python ile 100-999 aralığı tarandığında 153, 370, 371 ve 407 bulunur ve sayı 4 çıkar. Analiz için paylaşılan sayılar büyük olasılıkla bunlardır.

1.3 Anlamlı Değişken İsimleri

Çözüm ilk önce `B`, `C`, `D`, `E` adlarıyla söylendi. Hoca bu adların daha anlamlı olup olamayacağını sordu ve `yuzler`, `onlar`, `birler`, `kup` gibi adlar önerildi. Kuralı herkese şöyle anlattı: Değişkenin adı, sakladığı bilgiyi temsil etmelidir. Çok uzun adlar kullanın demiyor. Akış diyagramında yer darlığı yüzünden kısa adlar seçilebilir, programlarda yeri geldiğinde daha uzun adlar kullanılabilir. Önemli olan, bakan birinin adın ne anlama geldiğini anlamasıdır.

> **Hocanın vurgusu:** Bilgisayar mühendisliği kariyeriniz boyunca kullandığınız değişken isimlerine dikkat edin. Program yazarken de değişkenin adı, sakladığı bilgiyi temsil etmelidir.

1.4 Sorular

Soru: For döngüsünün içinde if kullanabiliyor muyuz? **Cevap:** Evet. For içinde başka bir for, while içinde for, for içinde if kullanılabilir. Bütün döngülerin içinde istediğiniz başka bir akış diyagramı gösterimini kullanabilirsiniz, bunda bir sakınca yoktur.

Soru: Bulunan sayıyı yazdırmadan yalnızca sayacı artırsak olmaz mı? **Cevap:** Sayıları şu anda bir yerde saklamıyoruz. Saklasaydık hepsini sonra bir arada gösterebilirdik. Birden fazla aynı tipte veriyi saklamak için birazdan göreceğimiz dizilerden faydalanacağız. Şimdilik sayıyı bulduğumuz anda yazdırıyor, aynı zamanda sayacı artırıyoruz.

Soru: `I` neden yazdırılıyor, analiz için mi? **Cevap:** Hayır. Sayılar yazdırılmazsa kullanıcı hangilerinin Armstrong sayısı olduğunu anlayamaz. Soru hem kaç tane olduğunu hem de sayıların kendisini istiyor. "Dört tane var" dedikten sonra hangi sayılar olduğunu da programı çalıştıran kişiye göstermek isteriz.

2. Algoritma 7: Arabanın Yakıt Tüketimi

İkinci problem gerçek hayattandı: Bir arabanın benzin tüketimi her saniye bilgi olarak verilmektedir. -1 değeri verildiğinde araba sürüşünü tamamlamış demektir. Her saniye için ortalama yakıt tüketimini ve sürüş ortalamasını ekrana yazdıran algoritmanın akış diyagramı çizilecektir.

2.1 Problemin Netleştirilmesi

Girdi sürekli gelir ve her turda alınır. O saniyeye kadarki ortalama ekrana yazılır. -1 girildiğinde sürüş biter ve son olarak sürüş boyuncaki ortalama yazdırılıp akış tamamlanır.

Soru: Saniyeler için ayrı bir şey yapacak mıyız? **Cevap:** Her turda bir saniye ilerliyormuş gibi düşünebilirsiniz.

Soru: Girdileri programın başında mı, döngü içinde mi alacağız? **Cevap:** Bilgi her saniye verildiğine göre her turda almalısınız.

Soru: Sürüş ortalaması ne demek? **Cevap:** Bütün sürüş boyunca gerçekleşen ortalamadır. Değerler saniye bazında verildiğine göre bütün sürüş için hesaplanır.

Soru: Bize toplam benzin mi veriliyor, yoksa o saniyede kullanılan miktar mı? **Cevap:** Soruda benzin tüketiminin her saniye verildiği yazıyor, yani saniye başına kullanılan benzin. Bunları toplayıp saniyeye bölerek ortalamayı buluruz.

> **Hocanın vurgusu:** Derste verilen süreler çok değerlidir. Doğruyu anlatıp geçmek mümkündür, ama önemli olan bunun üzerine kafa yorup yapmaya çalışmanızdır. "Ben düşündüm, böyle yapılır" deyip çizmemek yeterli değildir, çizmeniz gerekir.

2.2 For Değil While

Çözümü anlatan öğrenciler for ile başlamayı denedi. "Adım sayısı belli değil ama…" cümlesini hoca net biçimde kesti.

> **Hocanın vurgusu:** Adım sayısı belli değilse "aması" yoktur. For olmayacak demektir, while olacak demektir.

Koşul yazılırken bir öğrenci önce `BM ≠ 1` dedi, sonra `BM ≠ -1` diye düzeltti. Burada `BM`, benzin miktarı için seçilmiş bir addı. Hoca bu tür küçük hataların çok yapıldığını ve programı yanlış çalıştırdığını, canlı derste hata yapmanın sakıncası olmadığını, önemli olanın düzeltmek olduğunu söyledi. Koşullarda negatif değer kullanılabilir.

2.3 Akış Diyagramında Tanımlama Yoktur

Bir çözümde döngüden önce `toplam = 0` ve `x = 0` yazılmıştı. Hoca önce `x = 0` demeyi gerektiren bir durum olmadığını söyledi: Bize verilecek girdileri tanımlamak zorunda değiliz.

> **Hocanın vurgusu:** Bir değişkeni kullanmaya başladığınız yerden itibaren kullanabilirsiniz. Akış diyagramında tanımlama diye bir şey yoktur.

Öğrenci bunun üzerine asıl derdini söyledi: Koşul `x >= 0` ise ve yukarıda `x` ile ilgili bir şey yoksa döngüye ilk başta girilir mi? Hoca bu itirazı kabul etti. Şimdilik `x`'e 0 verilerek döngüye girilebilir. Başka bir yolu da C programlama dilinde göreceğimiz do-while kullanmaktır, yani önce işi yapıp sonra kontrol etmek. Henüz onu yapmadığımız için bu aşamada `x = 0` yazıldı.

> **Ek bilgi (derste anlatılmadı):** Hocanın "önce işi yapmak, sonra kontrol" sözüyle kastettiği, büyük olasılıkla do-while gövdesinin koşula bakılmadan en az bir kez çalışmasıdır. While ise koşulu gövdeden önce denetler, bu yüzden gövde hiç çalışmayabilir.

2.4 İlk Tasarım ve −1 Sorunu

Sınıfın birlikte kurduğu ilk tasarım şöyleydi (sayaç 1'den başlatılmıştı):

1. START.
2. `count = 1`, `toplam = 0`, `x = 0`. Kullanıcıya "araba sürüşü tamamladığında −1 giriniz" mesajı verilir.
3. While `x >= 0`:
   - Tüketim değeri istenir ve `x` okunur.
   - `count = count + 1`, `toplam = toplam + x`, `ort = toplam / (count - 1)`.
   - `ort` yazdırılır.
4. Döngüden çıkınca sürüş ortalaması yazdırılır.

Tasarımı anlatan öğrencinin kendisi sorunu da gördü: Kullanıcı −1 girdiğinde gövdedeki işlemler bir kez daha yapılır ve −1 girilmiş haldeki yanlış bir ortalama ekrana basılır. Onun çözüm önerisi, `x` okunduktan sonra bir if ile −1 olup olmadığına bakmak ve −1 ise işlemleri yaptırmamaktı.

Sınıftan başka fikirler de geldi:

- `x` girdisini gövdenin sonunda almak. Bir öğrenci bunun ilk turda değer olmadığı için hata vereceğini düşündü, hoca bu itiraza katılmadığını söyledi.
- Sohbet ortamından gelen öneri: İlk değeri bir kez döngünün dışında almak, sonraki değerleri gövdede hesaptan sonra almak.
- Koşulu `x >= 0` yerine `x ≠ -1` yazmak. Hoca bunun aynı noktaya geleceğini söyledi, çünkü şu anki koşulda da -1'de döngüden çıkılıyor.

2.5 Düzeltilmiş Tasarım

Hoca, bu sorunlarla karşılaşmamak için tasarımı değiştirdi. Araba sürüşe başladığında mutlaka bir değer gelecektir. Bu yüzden ilk değer döngüden önce okunur, yeni değer ise gövdenin en sonunda okunur. Böylece -1 geldiğinde hesap yapılmadan önce koşul kontrol edilir ve döngüden çıkılır. Sürüş sonu için ayrıca hesap gerekmez: Bir öğrencinin söylediği gibi, en son bulunan ortalama zaten istenen değerdir. Dersin devamındaki düzeltmelerle son hali şöyledir:

1. START.
2. `count = 0`, `toplam = 0`.
3. "İlk tüketim değerini giriniz" yazdırılır, `X` okunur.
4. `ort = X`.
5. While `X >= 0`:
   - `count = count + 1`
   - `toplam = toplam + X`
   - `ort = toplam / count`
   - `ort` yazdırılır.
   - "Tüketim değerini giriniz" yazdırılır, `X` okunur.
6. Döngüden çıkınca "Sürüş ortalaması" ve `ort` yazdırılır.
7. STOP.

Soru: Gövdenin sonunda neden yine "tüketim değerini giriniz" diyoruz? **Cevap:** Her adımda yeni saniyenin değerini almamız gerekir. Birinci saniyenin değerini döngüye girmeden aldık. Ortalamayı hesaplıyor, sonra ikinci saniyenin, sonra üçüncü saniyenin değerini alıyoruz. Döngüdeki adım sayısı saniyeye denk düşer, saniye için ekstra bir şey yapmanız gerekmez.

2.6 Analiz

Bir öğrenci, hocalarının çözümün yanına hemen analiz yapmayı öğütlediğini hatırlattı. Hoca da bunu onaylayarak analizi hemen yaptı. Analiz, o andaki tasarımla yani `count = 1` ve `ort = toplam / (count - 1)` ile yapıldı. Girilen değerler 5, 6, 7 ve -1'di. Hoca bu değerleri rastgele seçtiğini sonradan belirtti.

Okunan `X`	`X >= 0`	`count`	`toplam`	`ort`
5	evet	2	5	$5/1 = 5$
6	evet	3	11	$11/2 = 5{,}5$
7	evet	4	18	$18/3 = 6$
-1	hayır	döngüden çıkılır		

Çıkışta `ort` içinde 6 vardır ve sürüş ortalaması 6 olarak yazdırılır. Yanlış bir ortalama basılmamıştır.

2.7 Sayacın Başlangıç Değeri

Bir öğrenci, sayaç 0'dan başlatılırsa `count - 1` işlemine gerek kalmayacağını söyledi. Hoca dikkati buraya çekti: Bir yerde sayacı bir artırıyor, başka bir yerde bir azaltıyoruz. Yalnız sayacı 0 yapmak yetmez, `ort` satırındaki `count - 1` de mutlaka silinmelidir. Sonuç `ort = toplam / count` olur.

Alternatif de konuşuldu: Sayaç 1'den başlayacaksa işlem sırası değişir. Önce ortalama hesaplanır, sonra sayaç artırılır. Yani `toplam = toplam + X`, `ort = toplam / count`, `count = count`

+ 1. Hoca ikisinin de olacağını söyledi. Aynı 5, 6, 7, −1 girdisiyle üç sürüm de 5, 5,5 ve 6 ortalamalarını verir (ara adım eklendi). Sayaç 0'dan başlayan sürümde `count` değerleri 1, 2, 3 olur.

2.8 `ort` Değişkeninin İlk Değeri

Analiz sırasında hoca `ort` için ilk değer gerekip gerekmediğini sordu. Bir öğrencinin cevabı, ortalamaya önce atama yapılıp sonra kullanıldığı için sorun çıkmayacağıydı. Hoca bu cevabı o an onayladı. Fakat daha sonra bir soru bunu yeniden açtı.

Soru: Döngünün içine hiç girmeseydi, ortalamanın toplam bölü count olduğunu bilir miydi? Onu döngüde hesaplıyoruz. **Cevap:** Güzel bir soru. Döngüye girmeden çıkılan durum için `ort`'un döngüden önce bir değeri olması gerekir. `ort = 0` demek yerine `ort = X` yazalım, o zaman sorun kalmaz.

Hoca önce bunu "bir saniye çalışıp durduysa" diye anlattı, ardından kendisi düzeltti: İlk değer 0 ya da 1 girilse de döngüye girilir. Döngüye girilmeyen tek durum ilk değerin −1 girilmesidir ve bu, arabanın hiç çalışmadığı anlamına gelir. O zaman ekranda ortalama −1 görünür ve arabanın çalışmadığını anlarız. Bunlar istisnai durumlardır (exceptional case).

2.9 Koşul Neden `X >= 0`?

Soru: Koşula `X ≠ -1` dememek için özel bir sebep var mı? **Cevap:** `X ≠ -1` dersek ve araç yanlışlıkla −5 gibi başka bir negatif değer üretirse döngü devam eder. Bu şekilde uygulanabilir, ama gerçek hayatta normalde pozitif değerler girileceğini kabul ettiğimiz için ikisini de yazabilirdik, yanlış olmazdı.

2.10 While Seçimi ve Gerçek Hayat Bağlantısı

While kullanmamızın sebebi, sürüşün ne zaman tamamlanacağını bilmememizdir. 10 saniye de sürebilir, 50 saniye de, 3-5 dakika da, 1 saat de. Hoca, bu basit çözümün araçlarda ortalama yakıt tüketimini gösteren çözümle aynı olduğunu söyledi. Arka plandaki fizik ve mekanik kısmını hesaba katmıyoruz, `X`'in bize verildiğini varsayıyoruz. Gerçekte hız, yokuş, devir, sürtünme kuvveti ve aracın yükü tüketimi etkiler, ama sonuçta borudan geçen benzin miktarını ölçen bir sensör vardır. Bu miktara bağlı olarak anlık tüketimi görürsünüz, saniyeler ilerledikçe de ortalama tüketimi görürsünüz.

Soru: O saniyedeki ortalama için `X / count` yazmamız gerekmez mi? **Cevap:** Hayır. Tüketim saniyelik verilir. Onları topluyor, o zamana kadar geçen süreye bölüyorsunuz. Son bir saniyenin ortalaması olmaz, o zaten bir saniyelik tüketimdir.

Soru: Bilgi her saniye değil de 10 saniyede bir ya da dakikada bir gelseydi neyi değiştirirdik? **Cevap:** Burada her turun bir saniyeye karşılık geldiğini varsayıyoruz. Program yazarken zaman bekletme komutuyla saniyede bir bilgi alınması sağlanabilir. Araçta da bilgi belli periyotlarla verilir. Dakikada bir bilgi gelirse saniyelik ortalama veremezsiniz, dakikalık ortalama verirsiniz. Saniyelik bilgi alırken dakikalık ortalama da vermek isterseniz, içeriye bir if koyup `count` 60 olduğunda "dakikalık ortalamanız şudur" diye mesaj verebilirsiniz.

> **Ders sonrası not:** Hoca burada "if'in içerisinde bir if" dedi. Bu akışta dış if yoktur, büyük olasılıkla while kastedildi. Ayrıca `count = 60` koşulu yalnızca bir kez sağlanır. Mesajın her dakika tekrarlanması isteniyorsa büyük olasılıkla `count MOD 60 = 0` gibi bir koşul gerekir.

Soru: Analizdeki `X` değerleri gelişigüzel mi verildi? **Cevap:** Analizde evet, rastgele verdik ve hızla, araçla bir ilgisi yok. Algoritma açısından ise `X`'i gelişigüzel düşünmeyin. Günlük hayatta bu değeri aracın ürettiği bir ölçüm olarak düşünün.

3. Sabitler, Değişkenler ve Diziler

Hoca sıradaki soruya, final sınav notlarıyla ilgili bir probleme başlamıştı. Sonra önce konuya giriş yapmanın daha mantıklı olacağını söyleyip sabitler, değişkenler ve dizilere geçti.

3.1 Değişken ve Sabit

`a = 5` gibi bir atamada `a` değişken, `5` ise sabittir (constant). Bu değerin tam sayı olması gerekmez. `x = 3.2` yazıldığında `x` kesirli sayı değişkeni, `3.2` kesirli sayı sabitidir. Yani tipler farklı olabilir: Biri tam sayı değişkeni ve tam sayı sabiti, diğeri kesirli sayı değişkeni ve kesirli sayı sabitidir.

3.2 Dizi Nedir?

Bir değişken yalnızca bir değer saklar. Aynı tipte birden fazla değer saklamak istiyorsak dizi yapısını kullanırız.

Soru: Dizi dediğimiz liste mi oluyor? **Cevap:** Dizi, aynı tipteki elemanları bellekte birlikte, arka arkaya saklamak için kullandığımız bir veri yapısıdır.

Şu anda konuştuğumuz tek boyutlu dizilerdir (one-dimensional array). İki boyutlu olanları (two-dimensional) matris olarak adlandırılır. Programlama dilinde üç boyutlu, dört boyutlu diziler de tanımlanabilir. Bunlar ileride görülecek, başlangıç tek boyutlu dizilerledir.

Diziyi kullanırken köşeli parantezlerden faydalanırız. Köşeli parantez içine erişmek istediğimiz elemanın bilgisini, yani indisini (index) yazarız. Hoca 7 elemanlı bir dizide `dizi[7]` yazmanın 7. elemana erişmek anlamına geldiğini söyledi. Bu ifade, biraz sonra anlatılan 1'den başlayan algoritmik gösterim için geçerlidir.

3.3 İndisler: C ile Akış Diyagramı

Hocanın örnek dizisi 7 elemanlıydı: 5, 1, 4, −1, 0, 8, 9. C programlama dilinde dizinin ilk gözü `dizi[0]` diye adlandırılır, sonraki gözler birer birer artar. Bu dizinin ilk elemanı 5, son elemanı 9, eleman sayısı (number of elements) ise 7'dir.

Gösterim	İlk eleman	Son eleman	N elemanlı dizide aralık
C programlama dili	`dizi[0]`	`dizi[6]`	`dizi[0]` ... `dizi[N-1]`
Akış diyagramı (algoritmik gösterim)	`dizi[1]`	`dizi[7]`	`dizi[1]` ... `dizi[N]`

Hoca, algoritmik gösterimde sıfırıncı gözden `N-1`. göze kadar erişmek zorunda olmadığımızı, 1'den 7'ye kadar da erişebileceğimizi söyledi. C'de ise dizi bellekte `dizi[0]`'dan `dizi[6]`'ya kadar saklanır. Önemli olan, başlangıç indisini ne kabul ettiğinizdir. İlk etapta çözülecek sorularda dizinin 1'den başlayıp `N`'e kadar gittiği kabul edilecektir.

Soru: Dizilerde ilk terimi hep 1 olarak mı kullanacağız, 0 olarak kullanabilir miyiz? **Cevap:** Şimdilik 1 olarak kullanın. C programlamaya geçtiğinizde 0 kullanacaksınız. Akış diyagramlarında 1 olarak kabul edebilirsiniz, herhangi bir sıkıntı yok.

3.4 Akış Diyagramında Dizi Tanımlanmaz

Algoritma 8 çözülürken bir öğrenci, `A` dizisini diğer değişkenler gibi önce tanımlamak gerekip gerekmediğini sordu. Hoca, değişkenleri nasıl tanımlamıyorsak diziyi de akış diyagramında tanımlamayacağımızı söyledi. Tanımlama, program yazarken yapılır. C programlarında `int A[100];` benzeri bir tanımla 100'lük bir alan açılır.

4. Dizilerin Bellekte Saklanması

Hoca bir bellek (memory) şeması çizdi. Bellekte bir yerde `a` değişkeni vardır ve 5 değerini saklar. Başka bir yerde `x` değişkeni vardır ve 3.2 değerini saklar. Aralarında ne olduğunu bilmeyiz. Dizi elemanları ise sırayla yan yana yerleşir.

4.1 Adres Hesabı

Dizinin başlangıç adresi `0x1000` olsun. Saklanan değerler tam sayı (integer) olsun ve bu mimaride bir tam sayı 4 byte yer kaplasın. Soru şudur: Sonraki elemanlar hangi adreslerdedir?

4 byte yer kaplandığı için her eleman bir öncekinden 4 ileride başlar. 5 değeri `1000`, `1001`, `1002` ve `1003` adreslerini işgal eder. 1 değeri `1004`, `1005`, `1006` ve `1007` adreslerini işgal eder. Bir sonraki elemanın adresi `1008`'dir.

Hoca -1 değerinin, yani dizinin 4. elemanı `dizi[3]`'ün adresini sordu. Bu noktada bir öğrenci kafasının karıştığını söyledi.

Soru: Bu bin sayısı ondalık sistemde değil mi? **Cevap:** Hayır, onaltılık (hexadecimal). Başında `0x` var. `0x` gördüğünüzde bu sayı onaltılıktır.

Bu yüzden `0x1008`'den sonraki adres `0x100C` olur. 0 değerinin adresi sorulduğunda, başlangıçtan 16 ileride olduğu için `0x1010` bulunur. Tablonun tamamı aşağıdadır. Son satır derste söylenmedi (ara adım eklendi).

İndis (C)	Değer	Adres
`dizi[0]`	5	`0x1000`
`dizi[1]`	1	`0x1004`
`dizi[2]`	4	`0x1008`
`dizi[3]`	-1	`0x100C`
`dizi[4]`	0	`0x1010`
`dizi[5]`	8	`0x1014`
`dizi[6]`	9	`0x1018`

> **Ek bilgi (derste anlatılmadı):** `0x100C` sonucunun arkasındaki kural büyük olasılıkla şudur: Onaltılık sistemde basamaklar 0-9 ve A-F'dir, A = 10, B = 11, C = 12, D = 13, E = 14, F = 15. Bu yüzden 8 + 4 = 12, tek basamakta C ile yazılır. C + 4 = 16 ise basamağa sığmaz ve bir üst basamağa taşar, böylece `0x1010` elde edilir.

> **Ek bilgi (derste anlatılmadı):** Tablodaki düzen büyük olasılıkla genel bir formülün örneğidir: $\text{adres}(\text{dizi}[i]) = \text{başlangıç adresi} + i \times \text{eleman boyutu}$. Örneğin bu dizide son eleman için $\texttt{0x1000} + 6 \times 4 = \texttt{0x1018}$ bulunur.

4.2 Dizi ile Ayrı Değişkenlerin Farkı

Dizinin elemanları bellekte sırayla, arka arkaya saklanır. Aralarına başka bir şey girme ihtimali yoktur. Hoca bu özelliğe kesintisiz (continuous) dedi. `a` ve `x` değişkenleri ise belleğin farklı bölgelerinde saklanabilir. Şemadaki bölümleri, bellekte farklı adres blokları olarak düşünebilirsiniz.

> **Hocanın vurgusu:** Burada önemli olan şudur: Dizilerde saklanan değerler bellekte kesintisiz, arka arkaya olacak şekilde saklanır.

Soru: Bilgisayar bu değere erişmek için adresini bilmek zorunda. Adres için de ayrı bir alan tutuyor mu, bellekte nasıl erişiyor? **Cevap:** Değişkenleri tanımladığınızda, program derlenirken değişkenlerin hangi adreste olduğu bilgisi saklanır. Siz değişkeni çağırdığınızda o bellekteki adrese yönlendirilirsiniz. Bu, her değişken için bir kez saklanan bir değerdir.

5. Algoritma 8: Başarılı Öğrencilerin Sırasını Bulmak

Dizilerle çözülen ilk problem şuydu: N kişiden oluşan bir sınıfta final notları açıklanmıştır. Ortalamanın altında kalan öğrenciler başarısız, ortalamaya eşit ve üstünde olan öğrenciler başarılı kabul edilmektedir. Başarılı öğrencilerin kaçıncı sırada olduğu bilgisini bulan algoritmanın akış diyagramı çizilecektir.

5.1 Örnek Veri ve Kağıt Üzerinde Çözüm

Hoca örnek olarak 20, 100, 80, 70, 60, 50, 90 notlarını verdi. Burada `N` 7'dir, ama `N`'i her zaman değişken olarak almamız gerekir. Hoca önce bu işi kağıt üzerinde nasıl yapacağımızı sordu: Hepsini toplarız, kaç kişiyse o sayıya böleriz ve ortalamayı buluruz. Sonra her notu ortalamayla karşılaştırıp altında mı üstünde mi olduğuna bakarız. Hoca bunun basit bir ortalama bulma ve ortalamanın altında ya da üstünde yer alma problemi olduğunu, farklı versiyonlarının günlük hayatta görüldüğünü söyledi.

Derste bu verinin sonucu hesaplanmadı. Adımlar şöyledir (ara adım eklendi):

- Toplam: $20 + 100 + 80 + 70 + 60 + 50 + 90 = 470$
- Ortalama: $470/7 \approx 67{,}14$
- Ortalamanın altında kalanlar: 20 (1. sıra), 60 (5. sıra), 50 (6. sıra)
- Ortalamaya eşit ya da üstünde olanlar: 100 (2. sıra), 80 (3. sıra), 70 (4. sıra), 90 (7. sıra)

Algoritmanın yazdıracağı sıra numaraları 2, 3, 4 ve 7'dir.

5.2 Akış Diyagramı

Hoca daha fazla örnek çözebilmek için kullanıcıya verilecek mesajları yazmadı, ama sizin bunlara dikkat etmeniz gerektiğini söyledi. Çözüm üç döngüden oluşur: okuma, toplama, kontrol.

1. START.
2. `N` okunur.
3. For `I ← 1`, adım `1`, son değer `N`: `A[I]` okunur.
4. `toplam = 0`.
5. For `I ← 1`, adım `1`, son değer `N`: `toplam = toplam + A[I]`.
6. `ort = toplam / N`.
7. For `I ← 1`, adım `1`, son değer `N`: Eğer `A[I] >= ort` ise `I` yazdırılır. False kolunda bir şey yapılmaz.
8. STOP.

Notlar tek tek okunur. Birinci turda `A[1]`'e birinci öğrencinin, `I` 2 olduğunda ikinci öğrencinin, `I` 3 olduğunda üçüncü öğrencinin notu okunur. `A`, öğrencilerin notlarını sakladığımız dizidir. Hoca, `N` yerine `ogrenciSayisi` gibi anlamlı bir ad da kullanılabileceğini hatırlattı.

5.3 Tasarım Kararları

Ortalama döngünün içinde hesaplanmaz, çünkü her adımda ortalama hesaplamayı gerektiren bir durum yoktur. Döngüden çıktıktan sonra bir kez hesaplanır. Her seferinde hesaplanabilir, ama gerek yoktur.

Başarılı öğrencilere karar vermek için ortalamayı bilmek gerekir. Toplama döngüsü sırasında ortalama henüz bilinmediği için kontrol orada yapılamaz. Kontrol, ortalama bulunduktan sonra ayrı bir döngüde yapılır.

Koşul üzerine sınıfta uzun bir soru-cevap geçti. Sorulan şuydu: Üçüncü döngüde neyi neyle karşılaştıracağız? Notlar `A` dizisinde, ortalama `ort` değişkeninde saklanıyor. Dolayısıyla karşılaştırma `A` ile `N` arasında değil, `A[I]` ile `ort` arasındadır. Burada `N` yalnızca öğrenci sayısıdır.

Hoca koşulu tahtaya bilerek önce `A[I] > ort` diye yazdı ve tamamlanmasını bekledi. Bir öğrenci hatayı yakaladı.

> **Hocanın vurgusu:** Burada büyüktür derseniz sorunun istediği cevabı tam olarak vermiş olmazsınız. Ortalamanın altında kalanlar başarısız, eşit ve üstünde olanlar başarılıdır. Bu yüzden koşul `A[I] >= ort` olmalıdır.

Sohbet ortamından gelen bir fikir, başarılı öğrencileri ikinci bir dizide tutmaktı. Hoca bunu da yapacağımızı, ama ilk etapta yalnızca tek dizi üzerinde işlem yapmayı göstermek istediğini söyledi.

5.4 Sorular

Soru: "Kaçıncı sırada" derken puan sıralaması mı kastediliyor? **Cevap:** Hayır. Öğrencinin dizide hangi gözde olduğu, yani dizinin indisi isteniyor. Köşeli parantez içindeki kısma indeks (index), Türkçesiyle indis diyoruz. Öğrencinin notunun kaçıncı sırada olduğu değil, listedeki sıra numarası isteniyor. Birinci sıradaki öğrenci koşulu sağlıyorsa 1, ikinci sıradaki sağlıyorsa 2 yazdırılır.

Soru: `N` sıfır olursa hata oluşur, kontrol etmeyecek miyiz? **Cevap:** Boş bir sınıfın not ortalaması hesaplanmaz. Sensör örneğinde değer üretilirken hata olabilir, kullanıcı da hata yapabilir. Örneğin 5 yerine 6 verebilir, `N`'i eksi ya da sıfır girebilir. Bunlar kullanıcı hatalarıdır. Burada kullanıcının sınıftaki öğrenci sayısını doğru verdiğini kabul ediyoruz ve bu tür kontrolleri çok düşünmüyoruz.

Soru: Toplama işlemini okuma döngüsünün içinde yapsak hangisi daha hızlı olur? **Cevap:** Kesinlikle birleştirilmiş hali daha hızlı çalışır, çünkü şu anda N kez bir döngüde, N kez de başka bir döngüde dönüyorsunuz. Bu şekilde yapmak yanlış olmaz. Ayrı yazmamın sebebi okuma işini ve yapılan işi birbirinden ayırıp ilgili kısımların daha iyi anlaşılmasını sağlamaktır. Bu, tamamen öğretmek amaçlı bir yazım şeklidir.

6. Alıştırma: 365 Günün En Soğuk ve En Sıcak Günü

Son problem şuydu: Bir şehrin 365 günlük sıcaklık bilgisi verilmektedir. Yılın en soğuk ve en sıcak günlerini bulup sıcaklık değerleriyle birlikte ekrana yazdıran algoritmanın akış diyagramı çizilecektir.

6.1 Girdi

Dışarıdan yalnızca 365 günün sıcaklık değeri alınır. `N` okunmaz, çünkü sayı bellidir. Hoca sınıfa değerlerin nasıl olabileceğini sordu. Değerler daha hassas olabilir: 33,5 ya da 33,52 gibi kesirli, $-2{,}4$ gibi negatif, 0 gibi değerler görülebilir. Dizinin 1. gözünü 1 Ocak olarak düşünebilirsiniz.

6.2 Sınıftan Gelen Öneriler

- Bir değer seçip başka bir for ile bütün değerlerle kıyaslamak.
- Diziye okuduktan sonra bir değişken alıp dizideki her elemanla karşılaştırmak, büyükse değişkene atamak.
- Bir maksimum, bir minimum değişkeni alıp ikisine de ilk elemanı atamak, sonra diğerleriyle karşılaştırmak.
- Dereceleri sıfırdan büyük ve küçük diye gruplamak.
- Karşılaştırmayı aralık üzerinden yapmak, örneğin değerin 33,5 ile 7,2 arasında olup olmadığına bakmak.

Son iki öneri için hoca aynı gerekçeyi verdi: Sen gözle gördüğün için böyle düşünüyorsun, bilgisayar bunlardan hiçbirini görmüyor. Kimin sıfırdan büyük, kimin küçük olduğunu bilmiyoruz. Hepsini tek tek kontrol etmek gerekir. Hocanın benzetmesi çizgilerdi: İki çizgi çizdiğinizde, insan olarak bakıp paralel olduklarını aşağı yukarı söyleyebilirsiniz. Bilgisayara ise veri vermeniz gerekir.

6.3 Hocanın Anlattığı Çözüm Fikri

Hoca çözümü sözel olarak anlattı. İlk eleman alınır (33,5), geri kalan elemanlar onunla karşılaştırılır.

Maksimum için: Soru şöyle sorulur: 7,2, 33,5'ten büyük mü? Değil. 8,3 büyük mü? Değil. $-2{,}4$ büyük mü? Değil. Hepsi tek tek kontrol edilir. 34,0 görüldüğünde artık bu değer saklanır ve sonraki adımlarda soru "yeni değer 34,0'dan büyük mü?" olur.

Minimum için: 33,5 minimum kabul edilir. 7,2, 33,5'ten küçük mü? Evet, yeni minimum 7,2 olur. Üçüncü elemanda soru "8,3, 7,2'den küçük mü?" olur.

Sonunda elde maksimum olarak 34 kalıyorsa “34 ile 10. günde en yüksek sıcaklık ölçülmüştür”, −2,4 dizideki en küçük değerse “4. günde en düşük sıcaklık ölçülmüştür” diye çıktı verilir. Çıktıda gün de istendiği için değerin hangi günde olduğu bilgisi de gereklidir.

6.4 Başlangıç Değeri Neden İlk Eleman?

Soru: Dışarıdan başlangıç değeri vermek neden riskli? **Cevap:** Örneğin maksimuma 0 verip hepsinin ondan büyük olacağını kabul ederseniz, değerler o sınırın içinde kalmayabilir. Eksi 2, eksi 5, eksi 7 gibi değerler olabilir. Çok uç değerler de verebilirsiniz: Eksi bin derece olamaz, bin derece olamaz diyerek sınırı öyle belirlersiniz.

Bu problem için maksimum ve minimum başlangıcı olarak −100 ve 100 kullanılabilir, çünkü sıcaklıkta 100 derecenin ölçülme ihtimali normalde yoktur. Fakat başka maksimum ve minimum problemlerinde aralığı (range) bilemeyeceğiniz için sınır belirleyemeyebilirsiniz. Sohbet ortamından gelen bir uyarıyı da hoca onayladı: Maksimuma verdiğiniz değer dizideki tüm değerlerden büyükse sorun çıkar. Aynısı minimum için de geçerlidir.

> **Ders sonrası not:** Hoca Sibirya örneğinde “eksi 100 ölçüldü diyelim, eksi 500 olsun” dedi. Bu, sınırı geniş tutmak için söylenmiş bir varsayımdır. Fiziksel olarak −500 derece büyük olasılıkla mümkün değildir, çünkü mutlak sıfır yaklaşık −273 derecedir.
>
> **Hocanın vurgusu:** Her zaman ilk eleman maksimum ve minimum olarak kabul edilir, sonra geri kalan elemanlarla karşılaştırılır. Bu maksimum ve minimum bulma işi birçok problemde karşınıza çıkacaktır.

Bu soru alıştırma olarak verildi. Algoritma sözel olarak konuşuldu, akış diyagramının çizilmesi sınıfa bırakıldı. Sonraki derste bu sorunun üzerinden ilerlenecek.

Dersin Toparlaması

Bu derste döngülerle iki problem çözüldü. Armstrong sayılarında adım sayısı belli olduğu için for kullanıldı ve for içinde if ile hem yazdırma hem sayma yapıldı. Yakıt tüketiminde adım sayısı belli olmadığı için while kullanıldı. Asıl öğrenilen, girdiyi nerede okuduğumuzun sonucu değiştirdiğiydi: İlk değer döngüden önce, sonraki değerler gövdenin sonunda okununca −1 hesaba katılmaz. Sayacın başlangıç değeri ile ortalama formülünün birbirine bağlı olduğu da analizle gösterildi.

Dersin ikinci yarısında sabit, değişken ve dizi ayrımı yapıldı. Dizi aynı tipte birçok değeri bellekte arka arkaya saklar, indisi C’de 0’dan, akış diyagramlarında 1’den başlar. Algoritma 8’de okuma, toplama ve kontrol döngüleriyle dizinin ilk kullanımı görüldü. 365 günlük sıcaklık alıştırması ise ilk elemanla başlayan maksimum ve minimum bulma fikrini getirdi. Hoca sonraki derste dizilerle ilgili çok daha fazla ve daha ayrıntılı örnek çözüleceğini söyledi.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.